第一题：

```
vector<string> generateBoard(vector<int> &queens, int n) {
  auto board = vector<string>();
  for (int i = 0; i < n; i++) {
    string row = string(n, '.');
    row[queens[i]] = 'Q';
    board.push_back(row);
  }
  return board;
}

void backtrack(vector<vector<string>> &solutions, vector<int> &queens, int n,
               int row, unordered_set<int> &columns,
               unordered_set<int> &diagonals1, unordered_set<int> &diagonals2) {
  if (row == n) {
    vector<string> board = generateBoard(queens, n);
    solutions.push_back(board);
  } else {
    for (int i = 0; i < n; i++) {
      if (columns.find(i) != columns.end()) {
        continue;
```

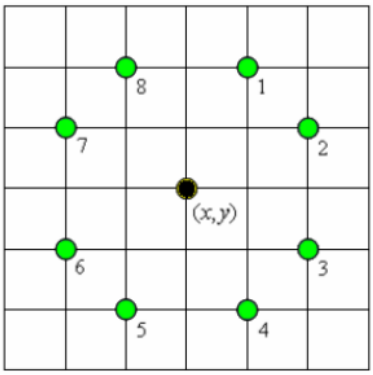
```
        }
        int diagonal1 = row - i;
        if (diagonals1.find(diagonal1) != diagonals1.end()) {
          continue;
        }
        int diagonal2 = row + i;
        if (diagonals2.find(diagonal2) != diagonals2.end()) {
          continue;
        }
        queens[row] = i;
        columns.insert(i);
        diagonals1.insert(diagonal1);
        diagonals2.insert(diagonal2);
        backtrack(solutions, queens, n, row + 1, columns, diagonals1, diagonals2);
        queens[row] = -1;
        columns.erase(i);
        diagonals1.erase(diagonal1);
        diagonals2.erase(diagonal2);
      }
    }
}

vector<vector<string>> solveNQueens(int n) {
    auto solutions = vector<vector<string>>();
    auto queens = vector<int>(n, -1);
    auto columns = unordered_set<int>();
    auto diagonals1 = unordered_set<int>();
    auto diagonals2 = unordered_set<int>();
    backtrack(solutions, queens, n, 0, columns, diagonals1, diagonals2);
    return solutions;
}
```

第二题：

将 backtrack 中 if(row==n){}中的生成解法替换为统计解法数目即可

第三题：



令 flag[0..8;0..8]表示棋盘，并初始化为 0，表示这些位置均未跳过。令(x0,y0)为起始位置，当前位置点为(x, y)。route 数组纪录访问路线。

```
JumpHorse(i)
1   for k←1 to 8 do
2     if x + dx[k]≥0 and x + dx[k] < 8 and y + dy[k]≥0 and y + dy[k] <8 then
3           route[i]←k
4           x ← x + dx[k];   y← y + dy[k]
5           if x=x0 and y=y0 then
6                 output(i); return
7           else
8                 if flag[x, y]=0 then
9                       flag[x, y]←1
10                      JumpHorse(i+1)
11          x←x - dx[k];   y←y - dy[k]
```

第四题：

```
int main()
{
    ans = n * m / 3 + 2;
    lim = n * m;
    for(int i=0;i<=n+1;++i)
        spy[i][0] = spy[i][m+1] = 1;
    for(int i=0;i<=m+1;++i)
        spy[0][i] = spy[n+1][i] = 1;
    search(1,1);
    printf("%d\n",ans );
    for(int i=1;i<=n;++i)
        for(int j=1;j<=m;++j)
            printf("%d%c",anx[i][j],j==m?'\n':' ' );
    return 0;
}
```

```
const int M = 32, go[5][2]={{0,0},{0,1},{0,-1},{1,0},{-1,0}};

int n,m;
int anx[M][M], ans;
int put[M][M], spy[M][M], tmp, spys = 0;
int lim1, lim2;

void search(int i,int j);
void puta(int x,int y,int i,int j)//在(x,y)陈列室放置一个机器人
{
    put[x][y] = 1;
    tmp++;
    for(int i=0; i<5; ++i)
        if(++spy[x + go[i][0]][y + go[i][1]]==1) spys++;

    search(i,j+1);

    put[x][y] = 0;
    tmp--;
    for(int i = 0; i < 5; ++i)
        if(--spy[x + go[i][0]][y + go[i][1]]==0) spys--;
}
void search(int i,int j)//访问（i,j）陈列室
{
    if(tmp>=ans) return;//剪枝1
    while(i<=n && spy[i][j]) //已放置的不再被搜索
    {
        ++j;
        if(j>m) ++i, j=1;
    }
    if(i>n)                    //更新答案
    {
        ans = tmp;
        memcpy(anx, put, sizeof(put));
        return;
    }

    int reach = spys + (ans-tmp)*5;
    if(reach <= lim) return;//剪枝2
    if((i==n&&j==m) || spy[i][j+1]==0) puta(i,j,i,j);
    if(i<n) puta(i+1,j,i,j);
    if(j<m && (spy[i][j+1]==0 || spy[i][j+2]==0)) puta(i,j+1,i,j);
```

第五题：

与题 4 类似，若某点周边已经被监视了，就不应该在该点放置机器人了